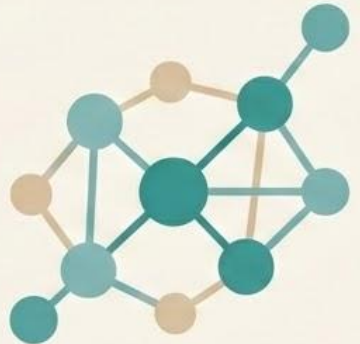


Foundations of LangGraph & Multi-Agent Systems



Session Objectives

- Understand Multi-Agent Systems
- Learn LangGraph fundamentals
- Explore state management
- Build intuition for agent collaboration



Why Do We Need Multi-Agent Systems?



Complex Task Handling

Breaks large, intricate problems into manageable, parallel workloads.



Specialization

Allows agents to focus on niche expertise for superior performance in specific domains.

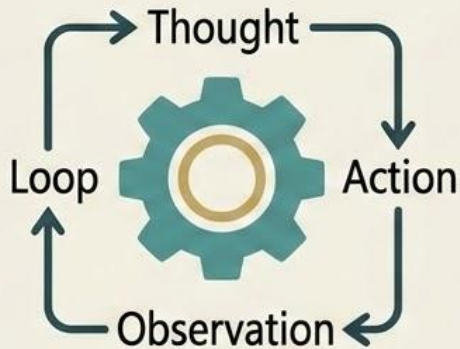


Collaboration

Enables independent agents to communicate, share knowledge, and achieve collective goals.

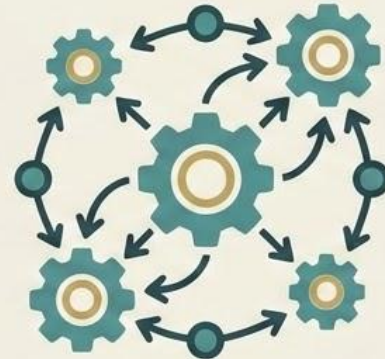
Single Agent vs Multi-Agent

SINGLE AGENT (REACT)



- * Monolithic structure
- * Serial processing
- * Sequential reasoning

MULTI-AGENT SYSTEM

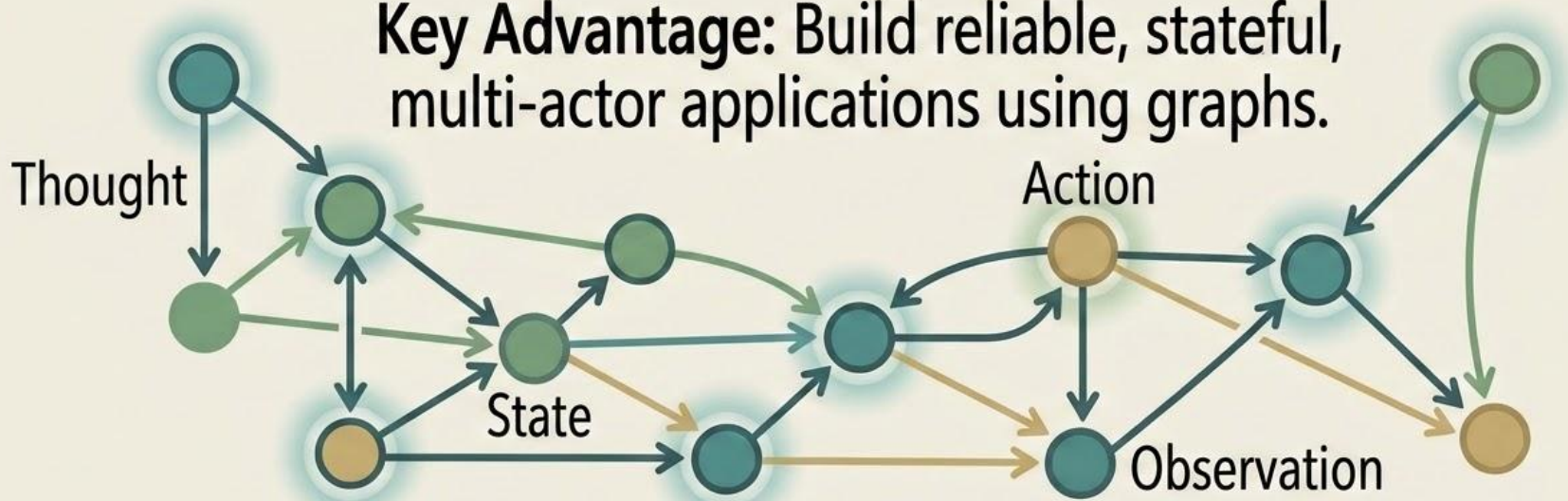


- * Decentralized team
- * Parallel execution
- * Diverse specialized expertise

What is LangGraph?

A sophisticated orchestration framework for building reliable, stateful LLM applications using graphs.

Key Advantage: Build reliable, stateful, multi-actor applications using graphs.



Core Concepts



Nodes

Individual agents or computational steps in your workflow.



Edges

Define the logic and transitions between nodes.



State

Shared information passed between steps in a workflow.

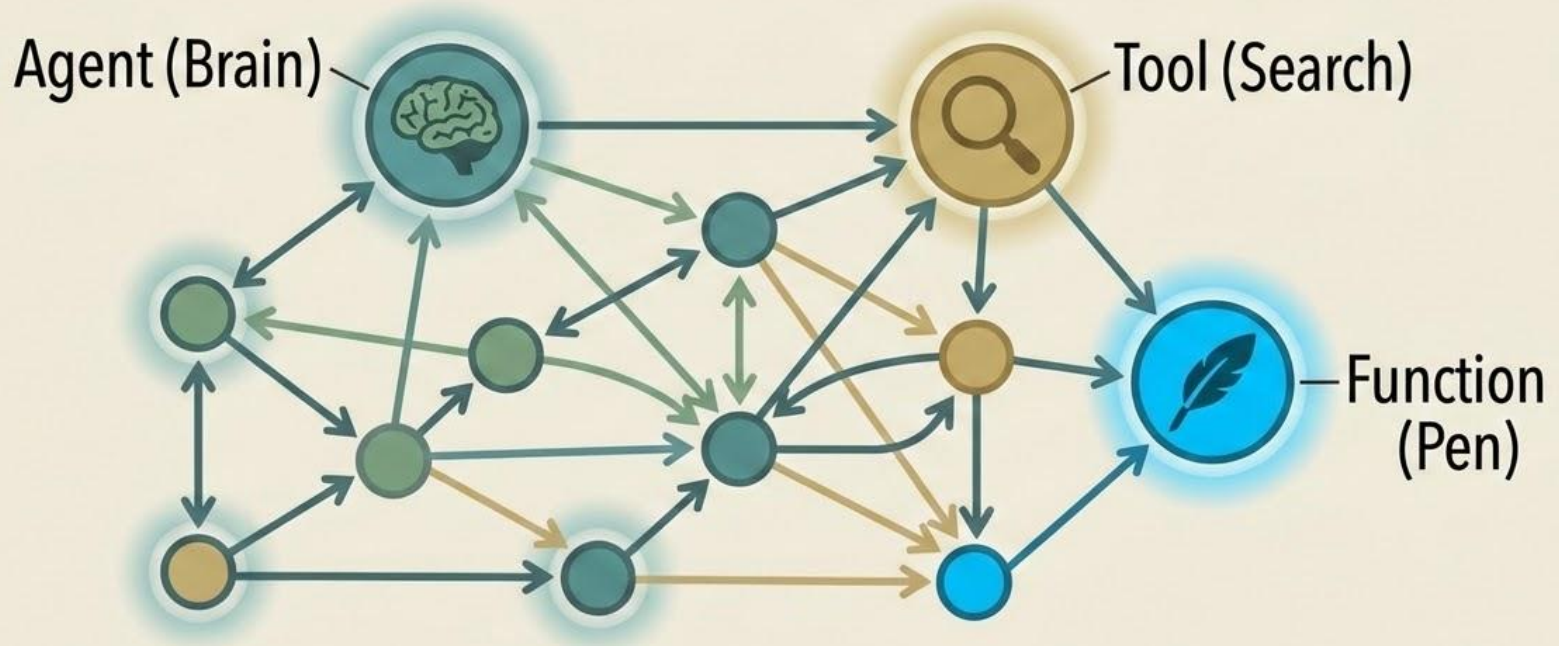


Graph

The overall structure orchestrating nodes, edges, and state.

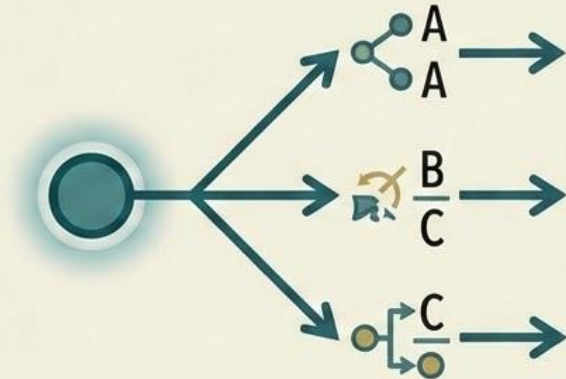
Nodes - The Building Blocks

Each node represents an agent, tool, or function



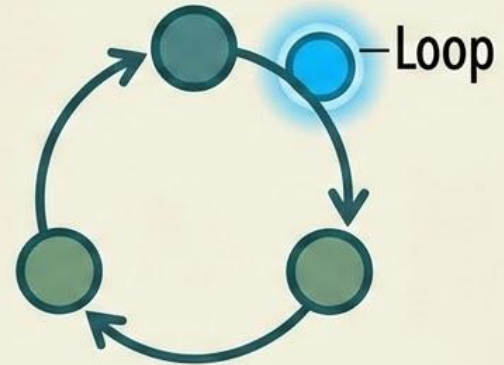
Edges - Connecting the Flow

Conditional Edges



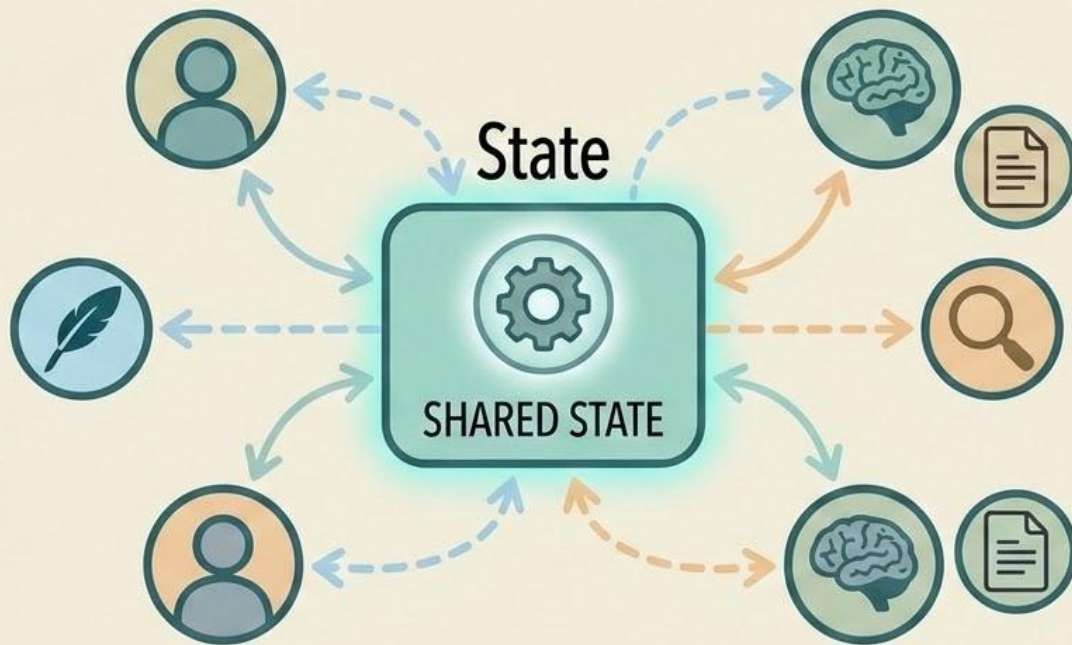
- Logic-based decision points.
- Transitions depend on workflow state.
- Enables complex, non-linear logic.

Cycles



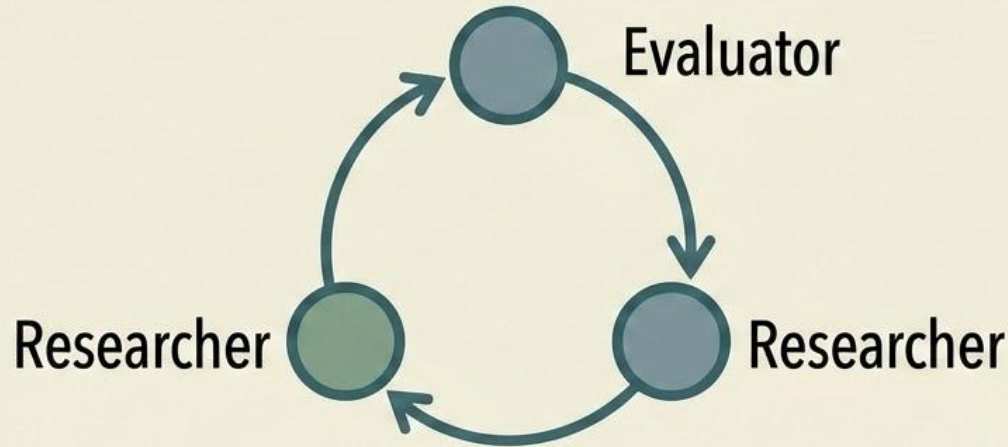
- Enables iterative processes.
- Returns flow to a previous step.
- Essential for optimization or loops.

State Management in LangGraph



Persistent memory shared across all agents

Cycles - Allowing Refinement



- Allows an agent (e.g., Researcher) to refine their work based on feedback from another agent (e.g., Evaluator).
- Enables multi-turn optimization and correction.
- Perfect for complex tasks requiring iterative quality control.

Real-World Use Cases



Research
Assistant



Customer
Support Team



Planning &
Execution



Content
Creation

Key Takeaways

- Cycles provide iterative refinement and error correction loops.
- Multi-agent systems enable collaborative complex task solving.
- Proven applications in diverse sectors like research, support, and planning.
- Iterative quality control is essential for complex automation.